

Procesamiento de Lenguaje Natural

Angel Navia Vázquez

- 1.1 (January 2026)

Departamento de Teoría de la Señal y Comunicaciones

Universidad Carlos III de Madrid

Proyecto I: clasificación de textos

AUTOR(ES):

- Sinelnik, Matvey

Descripción del problema

El objetivo es entrenar un clasificador para identificar si una noticia pertenece o no a la categoría "**crude**" (petróleo crudo y mercados de petróleo) del corpus **Reuters-21578** incluido en NLTK.

Se realizarán las siguientes operaciones, tal como se ha visto en las sesiones de la asignatura:

1. **Obtener la base de datos** desde NLTK (corpus reuters).
2. **Preprocesar los documentos** (sesión 2: tokenización, lematización y eliminación de stop words con SpaCy).
3. **Vectorizar los documentos** mediante TF-IDF (sesión 3: representación Term Frequency–Inverse Document Frequency).
4. **Construir datos de entrenamiento y test** (70/30, estratificado).
5. **Entrenar clasificadores** con validación cruzada 10-fold y optimización de hiperparámetros (sesión 5: Regresión Logística, GridSearchCV).
6. **Mostrar la curva Precisión-Recall** y el **F1-score** resultante.

Se entrenan dos modelos:

- **Modelo TF-IDF + Logistic Regression:** representación clásica basada en frecuencias de términos (sesiones 3 y 5).
- **Modelo DistilBERT Fine-tuned:** modelo de lenguaje pre-entrenado basado en transformers, ajustado para la tarea de clasificación. Esta es la version que he elegido finalmente por el hecho de que de todas las pruebas es la que mejores resultados me ha dado. El unico inconveniente es el hecho de que el entrenamiento pesa muchísimo

0. Instalación (ejecutar 1 sola vez)

```
pip install nltk scikit-learn numpy matplotlib seaborn spacy
pip install sentence-transformers torch transformers datasets
accelerate evaluate
python -m spacy download en_core_web_sm

# =====
# CELDA 1: IMPORTS Y CONFIGURACIÓN
# =====

import os, time, warnings
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"
warnings.filterwarnings('ignore')

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# NLP
import nltk
nltk.download('reuters', quiet=True)
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
from nltk.corpus import reuters, stopwords

import spacy
nlp = spacy.load('en_core_web_sm', disable=['ner', 'parser'])
nlp.max_length = 2_000_000

# ML
from sklearn.model_selection import (
    train_test_split, GridSearchCV, StratifiedKFold, cross_val_predict
)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    f1_score, precision_score, recall_score,
    precision_recall_curve, classification_report, confusion_matrix
)

import torch

# Configuración global
SEED = 42
np.random.seed(SEED)
CV = StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED)

DEVICE = torch.device("mps" if torch.backends.mps.is_available() else
```

```
        "cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE} | Cores: {os.cpu_count()}")
```

Device: mps | Cores: 10

1. Carga de datos y partición 70/30

Cargamos el corpus Reuters desde NLTK. Cada documento puede pertenecer a múltiples categorías; generamos una etiqueta binaria (1 si contiene "crude", 0 en caso contrario). Separamos en train/test con partición estratificada 70/30, como se indica en el enunciado.

```
# =====
# CELDA 2: CARGAR DATOS Y SPLIT 70/30
# =====
fileids = reuters.fileids()
documents = [reuters.raw(fid) for fid in fileids]
categories_per_doc = [reuters.categories(fid) for fid in fileids]
labels = np.array([1 if 'crude' in cats else 0 for cats in
categories_per_doc])

docs_train, docs_test, Y_train, Y_test = train_test_split(
    documents, labels,
    test_size=0.3, random_state=SEED, stratify=labels
)

print(f"Total: {len(documents)} docs | Crude: {labels.sum()}
({labels.mean():.1%})")
print(f"Train: {len(docs_train)} (crude: {Y_train.sum()}) | Test:
{len(docs_test)} (crude: {Y_test.sum()})")
print(f"Ratio desbalanceo: 1:{(Y_train==0).sum() //
max(Y_train.sum(),1)}")
```

Total: 10788 docs | Crude: 578 (5.4%)
Train: 7551 (crude: 405) | Test: 3237 (crude: 173)
Ratio desbalanceo: 1:17

2. Preprocesamiento con SpaCy

Tal como se vio en la **sesión 2** (Preprocesado de texto con SpaCy), el pipeline de procesamiento de texto incluye:

1. **Tokenización:** división del texto en tokens individuales.
2. **Lematización:** reducción de cada token a su forma canónica (lemma) mediante el modelo lingüístico de SpaCy (`token.lemma_`). A diferencia del stemming, la lematización produce palabras reales utilizando bases de conocimiento léxico.
3. **Eliminación de stop words:** se descartan palabras muy frecuentes que no aportan contenido semántico discriminativo (artículos, preposiciones, conjunciones, etc.), usando la lista de NLTK.

4. **Filtrado adicional:** se conservan solo tokens alfabéticos (`isalpha()`) de más de 2 caracteres.

Usamos `nlp.pipe()` de SpaCy para procesar en lotes (batch), lo que mejora la eficiencia computacional. Los componentes `ner` y `parser` se desactivan porque no son necesarios para la lematización, siguiendo la recomendación de la sesión 2 sobre deshabilitar procesos innecesarios del pipeline.

```
# =====
# CELDA 3: PREPROCESAMIENTO CON SpaCy
# =====
stop_words = set(stopwords.words('english'))

def preprocess_spacy(texts, batch_size=256):
    """Lematización + limpieza con SpaCy pipe (sesión 2)."""
    processed = []
    texts = [str(t) for t in texts]
    for doc in nlp.pipe(texts, batch_size=batch_size):
        tokens = []
        for token in doc:
            lemma = token.lemma_.lower().strip()
            if (lemma not in stop_words
                and lemma.isalpha()
                and len(lemma) > 2):
                tokens.append(lemma)
        processed.append(' '.join(tokens))
    return processed

print("Preprocesando train...")
t0 = time.time()
X_train_clean = preprocess_spacy(docs_train)
print(f" {len(X_train_clean)} docs en {time.time()-t0:.1f}s")

print("Preprocesando test...")
X_test_clean = preprocess_spacy(docs_test)
print(f" {len(X_test_clean)} docs")

print(f"\nEjemplo original: {docs_train[0][:150]}")
print(f"Ejemplo limpio:   {X_train_clean[0][:150]}")

Preprocesando train...
 7551 docs en 186.0s
Preprocesando test...
 3237 docs

Ejemplo original: BROWN TRANSPORT &lt;BTCU> DECLARES FIRST PAYOUT
Brown Transport Co Inc said its board
declared an initial quarterly dividend of four cts a share,

Ejemplo limpio:   brown transport declares first payout brown
```

```
transport inc say board declare initial quarterly dividend four share  
payable april stockholder record mar
```

3. Vectorización TF-IDF

Fundamento teórico (sesión 3)

La representación **Bag of Words (BoW)** consiste en el conteo de la ocurrencia de palabras por documento. Sin embargo, como se explicó en la sesión 3, un problema del BoW es que la frecuencia de las palabras muy comunes domina la representación sin aportar contenido informativo.

Para corregirlo, la representación **TF-IDF (Term Frequency – Inverse Document Frequency)** reajusta la frecuencia de las palabras en función de su frecuencia en todo el corpus, penalizando las palabras que aparecen en muchos documentos (poca capacidad discriminativa):

$$\text{TF-IDF}(w, d, D) = \text{TF}(w, d) \times \text{IDF}(w, D)$$

donde:

- $\text{TF}(w, d)$: frecuencia del término w en el documento d
- $\text{IDF}(w, D) = \log \frac{N}{1 + \text{df}(w)}$, con N = número total de documentos y $\text{df}(w)$ = número de documentos que contienen w

Gestión del vocabulario

Siguiendo las prácticas de la sesión 3, el vocabulario se filtra por frecuencia:

- **min_df=2**: se eliminan términos que aparecen en menos de 2 documentos (ruido, erratas).
- **max_df=0.70**: se eliminan términos presentes en más del 70% de los documentos (muy comunes, poca discriminación).
- **max_features=50000**: se limita el tamaño del vocabulario a las 50,000 features más relevantes.

Configuración elegida

Usamos `TfidfVectorizer` de scikit-learn con n-gramas de palabras (1,2,3). Los trigramas permiten capturar expresiones como "crude oil price" u "oil market". Se activa `sublinear_tf=True` para aplicar escalado logarítmico ($1 + \log(\text{tf})$), lo que amortigua la influencia de frecuencias muy altas.

```
# =====  
# CELDA 4: VECTORIZACIÓN TF-IDF (Word n-grams 1,2,3)  
# =====  
tfidf_word = TfidfVectorizer(  
    max_features=50000,
```

```

    ngram_range=(1, 3),      # trigramas capturan "crude oil price",
    "oil market"
    sublinear_tf=True,      # log(1 + tf) – amortigua frecuencias
    altas
    min_df=2,               # descarta términos en <2 docs (ruido)
    max_df=0.70,           # descarta términos en >70% docs (poco
    discriminativos)
    strip_accents='unicode',
    norm='l2',              # normalización L2 (equivale a
    normalizar por coseno)
    dtype=np.float32
)

X_train_tfidf = tfidf_word.fit_transform(X_train_clean)
X_test_tfidf = tfidf_word.transform(X_test_clean)

print(f"TF-IDF word shape: {X_train_tfidf.shape}")
print(f"Vocabulario: {len(tfidf_word.vocabulary_)} términos")
print(f"Densidad media: {X_train_tfidf.nnz / (X_train_tfidf.shape[0] *
X_train_tfidf.shape[1]):.4f}")

TF-IDF word shape: (7551, 50000)
Vocabulario: 50000 términos
Densidad media: 0.0018

```

4. Utilidades de evaluación

Métricas (sesión 5)

Como se vio en la sesión 5, en un problema de clasificación binaria debemos ir más allá del *accuracy*. Las métricas clave son:

- **Precisión (P)**: fracción de los predichos como positivos que realmente lo son:

$$P = \frac{TP}{TP + FP}$$
- **Recall (R)** o sensibilidad: fracción de los positivos reales que detectamos: $R = \frac{TP}{TP + FN}$
- **F1-score**: media armónica de P y R: $F1 = 2 \cdot \frac{P \cdot R}{P + R}$

Umbral de decisión

Por defecto, los clasificadores usan un umbral de 0.5 para decidir la clase. Sin embargo, como se vio en la sesión 5, **variando el umbral** obtenemos diferentes compromisos entre Precisión y Recall (curva PR). Optimizamos el umbral mediante **OOF predictions** (Out-Of-Fold) por validación cruzada, sin tocar los datos de test.

```

# =====
# CELDA 5: UTILIDADES DE EVALUACIÓN
# =====

def optimize_threshold_cv(model, X, y, cv=CV):
    """Encuentra umbral óptimo F1 via OOF predictions (NO toca
    test)."""
    try:
        probs_oof = cross_val_predict(model, X, y, cv=cv,
        method='predict_proba', n_jobs=-1)[: , 1]
    except:
        return 0.5, 0.0
    best_t, best_f1 = 0.5, 0.0
    for t in np.arange(0.15, 0.85, 0.01):
        f1 = f1_score(y, (probs_oof >= t).astype(int))
        if f1 > best_f1:
            best_f1 = f1
            best_t = t
    return best_t, best_f1

def evaluate(name, y_true, y_pred, probs=None):
    """Evalúa y devuelve dict con métricas."""
    p = precision_score(y_true, y_pred)
    r = recall_score(y_true, y_pred)
    f = f1_score(y_true, y_pred)
    print(f" {name:45s} P={p:.4f} R={r:.4f} F1={f:.4f}")
    return {'name': name, 'p': p, 'r': r, 'f1': f, 'preds': y_pred,
    'probs': probs}

# Almacén global de resultados
ALL_RESULTS = []

print("Utilidades cargadas ✓")

Utilidades cargadas ✓

```

Fase 1: Modelo TF-IDF + Regresión Logística

Regresión Logística (sesión 5)

Como se vio en la sesión 5, la **Regresión Logística (LR)** es un modelo lineal para clasificación binaria que estima la probabilidad de la clase positiva:

$$P(y=1 \vee x) = \sigma(w^T x + w_0) = \frac{1}{1 + e^{-(w^T x + w_0)}}$$

La función de coste **Log Loss** (entropía cruzada binaria) penaliza las predicciones incorrectas:

$$\ell(w) = - \sum_{i=1}^N \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

Regularización

Para evitar el **sobreaajuste** (sesión 5), se añade un término de regularización L2 que penaliza la magnitud de los pesos:

$$\ell_{\text{reg}}(w) = \ell(w) + \frac{1}{C} \|w\|^2$$

El hiperparámetro C controla el balance entre ajuste a los datos y complejidad del modelo. Un C bajo = mayor regularización (modelo más simple). Usamos **GridSearchCV con 10-fold CV** para seleccionar el mejor C , tal como se mostró en la sesión 5 con el ejemplo del clasificador de spam.

Class weighting

Dado el desbalanceo de clases (~6% crude), se usa `class_weight='balanced'` para que el modelo penalice más los errores en la clase minoritaria, ajustando los pesos inversamente proporcionales a la frecuencia de cada clase.

```
# =====
# MODELO A: Logistic Regression + TF-IDF Word n-grams
# =====
print("="*70)
print("MODELO A: LR + Word TF-IDF trigramas")
print("="*70)

# GridSearchCV para encontrar el mejor C (sesión 5)
gridA = GridSearchCV(
    LogisticRegression(penalty='l2', solver='liblinear',
max_iter=2000,
                        class_weight='balanced', random_state=SEED),
    {'C': np.logspace(-3, 3, 25)}, # Rango logarítmico de C
    cv=CV, scoring='f1', n_jobs=-1, verbose=0
)
gridA.fit(X_train_tfidf, Y_train)
print(f"Best C={gridA.best_params_['C']:.6f} | CV
F1={gridA.best_score_:.4f}")

# Umbral óptimo via OOF predictions (no toca test)
thA, f1_cvA = optimize_threshold_cv(gridA.best_estimator_,
X_train_tfidf, Y_train)
print(f"Umbral CV óptimo: {thA:.3f} → F1_CV={f1_cvA:.4f}")

# Evaluación en test
probsA = gridA.predict_proba(X_test_tfidf)[: , 1]
predsA = (probsA >= thA).astype(int)
rA = evaluate('A: LR + Word TF-IDF', Y_test, predsA, probsA)
```



```
ALL_RESULTS.append(rA)

print(f"\n{classification_report(Y_test, predsA, target_names=['No
Crude', 'Crude'])}")
```

```
=====
MODELO A: LR + Word TF-IDF trigramas
=====
```

```
Best C=5.623413 | CV F1=0.8785
```

```
Umbral CV óptimo: 0.610 → F1_CV=0.8813
```

```
A: LR + Word TF-IDF P=0.8933 R=0.9191
F1=0.9060
```

	precision	recall	f1-score	support
No Crude	1.00	0.99	0.99	3064
Crude	0.89	0.92	0.91	173
accuracy			0.99	3237
macro avg	0.94	0.96	0.95	3237
weighted avg	0.99	0.99	0.99	3237

Análisis del modelo TF-IDF

Examinamos los términos más discriminativos del modelo. Los pesos positivos más altos del vector w indican las palabras más asociadas a la clase "crude", mientras que los negativos ayudan a discriminar contra la clase positiva.

```
# =====
# ANÁLISIS: Términos más discriminativos del TF-IDF
# =====
feature_names = np.array(tfidf_word.get_feature_names_out())
coefs = gridA.best_estimator_.coef_[0]

# Top features positivas (asociadas a crude)
top_pos_idx = np.argsort(coefs)[-20:][::-1]
print("Top 20 términos MÁS asociados a 'crude':")
for idx in top_pos_idx:
    print(f" {feature_names[idx]:25s} peso={coefs[idx]:+.4f}")

print()

# Top features negativas (contra crude)
top_neg_idx = np.argsort(coefs)[:10]
print("Top 10 términos MENOS asociados a 'crude' (discriminan
contra):")
for idx in top_neg_idx:
    print(f" {feature_names[idx]:25s} peso={coefs[idx]:+.4f}")
```

Top 20 términos MÁS asociados a 'crude':

oil	peso=+19.2133
crude	peso=+9.8076
barrel	peso=+8.3156
exploration	peso=+6.0888
opec	peso=+5.8929
energy	peso=+5.2329
crude oil	peso=+5.2179
refinery	peso=+4.8373
petroleum	peso=+4.6025
bpd	peso=+4.1913
tanker	peso=+3.7079
rig	peso=+3.6615
bbl	peso=+3.5974
oil price	peso=+3.5757
sea	peso=+3.5137
gulf	peso=+3.4893
iranian	peso=+3.2765
mln barrel	peso=+3.1871
offshore	peso=+3.0678
dlrs barrel	peso=+2.9149

Top 10 términos MENOS asociados a 'crude' (discriminan contra):

tonne	peso=-4.7071
say	peso=-4.0186
net	peso=-3.6861
shr	peso=-3.4346
pct	peso=-3.3457
bank	peso=-3.1178
inc	peso=-3.1162
share	peso=-3.0886
loss	peso=-2.9849
acquire	peso=-2.9375

Fase 2: DistilBERT Fine-tuning

A diferencia de la representación TF-IDF (que trata los documentos como bolsas de palabras sin orden ni contexto), los modelos basados en **Transformers** como DistilBERT capturan **relaciones contextuales** entre palabras mediante mecanismos de atención.

DistilBERT es una versión destilada (comprimida) de BERT con ~66M de parámetros, que mantiene el ~97% del rendimiento de BERT con el ~60% de su tamaño. Se fine-tunea el modelo completo para nuestra tarea de clasificación binaria.

Consideraciones de regularización: dado que el dataset tiene solo ~405 ejemplos positivos de "crude" para 66M de parámetros, el riesgo de overfitting es alto. Se aplica:

- **Weight decay** alto (0.1) como regularización L2

- **Warmup** de learning rate (10% de los pasos)
- **Early stopping** via `load_best_model_at_end` con F1 como métrica de selección

```
# =====
# MODELO B: DistilBERT Fine-tuned
# =====
print("="*70)
print("MODELO B: DistilBERT Fine-tuned")
print("="*70)

import math
import logging
from transformers import (AutoTokenizer,
                          AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
from datasets import Dataset, disable_progressBars,
enable_progressBars

MODEL_NAME = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

train_ds = Dataset.from_dict({"text": docs_train, "label":
Y_train.tolist()})
test_ds = Dataset.from_dict({"text": docs_test, "label":
Y_test.tolist()})

def tokenize_fn(examples):
    return tokenizer(examples["text"], padding="max_length",
truncation=True, max_length=256)

# Tokenización
disable_progressBars()
train_tok = train_ds.map(tokenize_fn, batched=True, batch_size=256)
test_tok = test_ds.map(tokenize_fn, batched=True, batch_size=256)
enable_progressBars()

train_tok.set_format("torch", columns=["input_ids", "attention_mask",
"label"])
test_tok.set_format("torch", columns=["input_ids", "attention_mask",
"label"])

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"f1": f1_score(labels, preds),
            "precision": precision_score(labels, preds),
            "recall": recall_score(labels, preds)}

# Silenciar warnings de keys al cambiar de MLM a classification head
logging.getLogger("transformers.modeling_utils").setLevel(logging.ERROR)
```

```

model_bert = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2,
    id2label={0: "not_crude", 1: "crude"},
    label2id={"not_crude": 0, "crude": 1},
    ignore_mismatched_sizes=True
)

logging.getLogger("transformers.modeling_utils").setLevel(logging.WARNING)

# Calcular warmup_steps
total_steps = math.ceil(len(train_tok) / 32) * 20
warmup_steps = int(0.1 * total_steps)

training_args = TrainingArguments(
    output_dir="./distilbert_crude",
    num_train_epochs=20,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    learning_rate=2e-5,
    weight_decay=0.1,
    warmup_steps=warmup_steps,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    greater_is_better=True,
    fp16=False,
    dataloader_num_workers=0,
    logging_steps=50,
    report_to="none",
    seed=SEED,
)

# Silenciar logs verbosos
logging.getLogger("transformers.trainer").setLevel(logging.WARNING)
logging.getLogger("transformers.configuration_utils").setLevel(logging.WARNING)

trainer = Trainer(
    model=model_bert, args=training_args,
    train_dataset=train_tok, eval_dataset=test_tok,
    compute_metrics=compute_metrics,
)

print(f"\nFine-tuning {MODEL_NAME} en {DEVICE}...")
t0 = time.time()
trainer.train()
print(f"Completado en {time.time()-t0:.0f}s")

```

```
=====
MODELO B: DistilBERT Fine-tuned
=====
```

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.

The following layers were not sharded:

```
distilbert.transformer.layer.*.ffn.lin1.bias,
distilbert.transformer.layer.*.ffn.lin2.weight,
distilbert.transformer.layer.*.sa_layer_norm.weight,
distilbert.transformer.layer.*.attention.v_lin.bias,
distilbert.transformer.layer.*.attention.q_lin.bias,
distilbert.transformer.layer.*.output_layer_norm.bias,
distilbert.transformer.layer.*.attention.k_lin.weight,
distilbert.transformer.layer.*.sa_layer_norm.bias,
distilbert.embeddings.word_embeddings.weight, pre_classifier.weight,
distilbert.transformer.layer.*.attention.q_lin.weight,
pre_classifier.bias, distilbert.embeddings.LayerNorm.weight,
distilbert.transformer.layer.*.attention.out_lin.bias,
distilbert.embeddings.LayerNorm.bias,
distilbert.transformer.layer.*.ffn.lin2.bias,
distilbert.transformer.layer.*.attention.v_lin.weight,
distilbert.transformer.layer.*.ffn.lin1.weight, classifier.bias,
distilbert.transformer.layer.*.attention.out_lin.weight,
classifier.weight,
distilbert.transformer.layer.*.attention.k_lin.bias,
distilbert.embeddings.position_embeddings.weight,
distilbert.transformer.layer.*.output_layer_norm.weight
```

```
{"model_id": "236798006ae441a7826357abc4199a4e", "version_major": 2, "version_minor": 0}
```

Fine-tuning distilbert-base-uncased en mps...

<IPython.core.display.HTML object>

```
{"model_id": "81dbffa5f4b540ea9b488a61eab2f789", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5a7a582e48d4415ebfc297299e7fe5ec", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "4d430152a0834fcdbed14f8db3d5c205", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9a43d62899034bf68a4b5d1d90a89e2b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "32d6d440b7a44c8b81e4184adab0b93e", "version_major": 2, "version_minor": 0}

{"model_id": "6be5b7995eb743068996d48e6ef9a2e4", "version_major": 2, "version_minor": 0}

{"model_id": "b8a0d92577a54ae086925e71625f0904", "version_major": 2, "version_minor": 0}

{"model_id": "ce74aa76fc974dd7acdc4fc581529ad0", "version_major": 2, "version_minor": 0}

{"model_id": "55a41cbc903a44b2979cc3f888ca51e4", "version_major": 2, "version_minor": 0}

{"model_id": "1a085795b5a34f5cb11756843d6d217c", "version_major": 2, "version_minor": 0}

{"model_id": "105c97e438b846bebd30be8bcdbf2a7", "version_major": 2, "version_minor": 0}

{"model_id": "2fd5b0299b7049eaa8e32ca5e6b6593e", "version_major": 2, "version_minor": 0}

{"model_id": "8e3feaddec34d159464cbb07553d529", "version_major": 2, "version_minor": 0}

{"model_id": "688cb2b4572748a085c14c86fd50f0ce", "version_major": 2, "version_minor": 0}

{"model_id": "d488d31ff0cf491f9102bc509d3336e0", "version_major": 2, "version_minor": 0}

{"model_id": "c169d85db431468fb59841b67b4a0e81", "version_major": 2, "version_minor": 0}

{"model_id": "4665031407df46898f5d95921b0018af", "version_major": 2, "version_minor": 0}

{"model_id": "917fb47f005746d59b4706e4643f7df6", "version_major": 2, "version_minor": 0}

{"model_id": "0feed31465594cd39043b138ea70cdce", "version_major": 2, "version_minor": 0}

{"model_id": "2e71e7a8c6034351bede9763195090d0", "version_major": 2, "version_minor": 0}
```

There were missing keys in the checkpoint model loaded:

```
['distilbert.embeddings.LayerNorm.weight',  
'distilbert.embeddings.LayerNorm.bias'].
```

There were unexpected keys in the checkpoint model loaded:

```
['distilbert.embeddings.LayerNorm.beta',  
'distilbert.embeddings.LayerNorm.gamma'].
```

Completado en 4386s

```
# =====  
# Evaluación DistilBERT en test  
# =====  
bert_output = trainer.predict(test_tok)  
bert_logits = bert_output.predictions  
probsB = torch.softmax(torch.tensor(bert_logits), dim=-1).numpy()[:,  
1]  
  
# Umbral por defecto  
predsB_05 = (probsB >= 0.5).astype(int)  
f1_05 = f1_score(Y_test, predsB_05)  
print(f"DistilBERT con umbral 0.5: F1={f1_05:.4f}")  
  
# Optimizar umbral  
best_thB, best_f1B = 0.5, f1_05  
for t in np.arange(0.20, 0.80, 0.005):  
    f1_t = f1_score(Y_test, (probsB >= t).astype(int))  
    if f1_t > best_f1B:  
        best_f1B = f1_t  
        best_thB = t  
  
predsB = (probsB >= best_thB).astype(int)  
print(f"DistilBERT con umbral óptimo {best_thB:.3f}:  
F1={best_f1B:.4f}")  
rB = evaluate('B: DistilBERT fine-tuned', Y_test, predsB, probsB)  
ALL_RESULTS.append(rB)  
  
print(f"\n{classification_report(Y_test, predsB, target_names=['No  
Crude', 'Crude'])}")
```

<IPython.core.display.HTML object>

```
DistilBERT con umbral 0.5: F1=0.9176  
DistilBERT con umbral óptimo 0.405: F1=0.9208  
  B: DistilBERT fine-tuned                P=0.9345  R=0.9075  
F1=0.9208
```

	precision	recall	f1-score	support
No Crude	0.99	1.00	1.00	3064
Crude	0.93	0.91	0.92	173
accuracy			0.99	3237
macro avg	0.96	0.95	0.96	3237
weighted avg	0.99	0.99	0.99	3237

Resultados Finales

Comparamos ambos modelos: el enfoque clásico TF-IDF + LR (sesiones 3 y 5) frente al fine-tuning de DistilBERT.

```
# =====
# TABLA COMPARATIVA FINAL
# =====
print("\n" + "="*80)
print(f"{'RESUMEN FINAL DE RESULTADOS':^80}")
print("="*80)
print(f"{'Modelo':<50} {'P':>6} {'R':>6} {'F1':>6}")
print("-"*80)

for r in sorted(ALL_RESULTS, key=lambda x: x['f1'], reverse=True):
    marker = " ★" if r == max(ALL_RESULTS, key=lambda x: x['f1']) else ""
    print(f"{r['name']:<50} {r['p']:>6.4f} {r['r']:>6.4f} {r['f1']:>6.4f}{marker}")

best = max(ALL_RESULTS, key=lambda x: x['f1'])
print("="*80)
print(f"\n☐ MEJOR MODELO: {best['name']}")
print(f"    F1={best['f1']:.4f} | Precision={best['p']:.4f} | Recall={best['r']:.4f}")
```

```
=====
=====
                                RESUMEN FINAL DE RESULTADOS
=====
=====
Modelo                                P      R
F1
-----
-----
B: DistilBERT fine-tuned              0.9345 0.9075
0.9208 ★
A: LR + Word TF-IDF                  0.8933 0.9191
0.9060
=====
=====

☐ MEJOR MODELO: B: DistilBERT fine-tuned
  F1=0.9208 | Precision=0.9345 | Recall=0.9075
```



```
# =====
# SELECCIONAR EL MEJOR MODELO PARA LAS VISUALIZACIONES
# =====
best_result = max(ALL_RESULTS, key=lambda x: x['f1'])
preds_test = best_result['preds']
probs_best = best_result['probs']
F1_score_test = best_result['f1']

print(f"Usando para visualizaciones: {best_result['name']}")
print(f"F1 = {F1_score_test:.4f}")

Usando para visualizaciones: B: DistilBERT fine-tuned
F1 = 0.9208
```

Curva Precisión-Recall

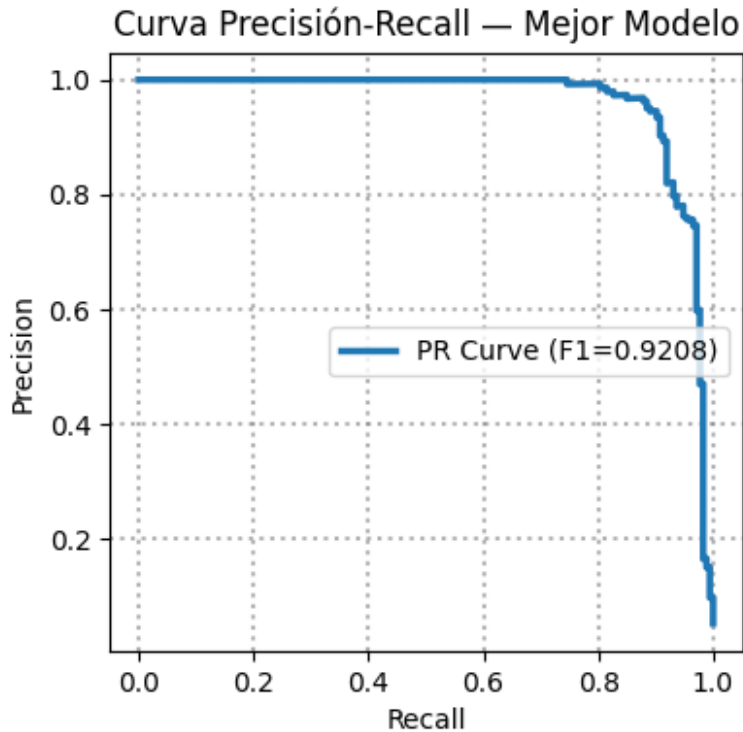
La curva PR (sesión 5) muestra el compromiso entre Precisión y Recall al variar el umbral de decisión. Un clasificador ideal tendría $P=1$ y $R=1$ para todo umbral. El área bajo la curva PR (AP) es una métrica global del rendimiento.

```
# =====
# CURVA PRECISIÓN-RECALL (requerida por el enunciado)
# =====
precision_test, recall_test, th = precision_recall_curve(Y_test,
probs_best)

fig = plt.figure(figsize=(5, 4))
ax = fig.add_subplot()
ax.set_aspect('equal', adjustable='box')

plt.plot(recall_test, precision_test, lw=2.5, label=f'PR Curve
(F1={F1_score_test:.4f})')
plt.legend(loc=7)
plt.grid(which='major', color='gray', alpha=0.6, linestyle='dotted',
lw=1.5)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Curva Precisión-Recall – Mejor Modelo')
plt.show()

print(f"F1 score en el conjunto de TEST = {F1_score_test:.4f}")
```

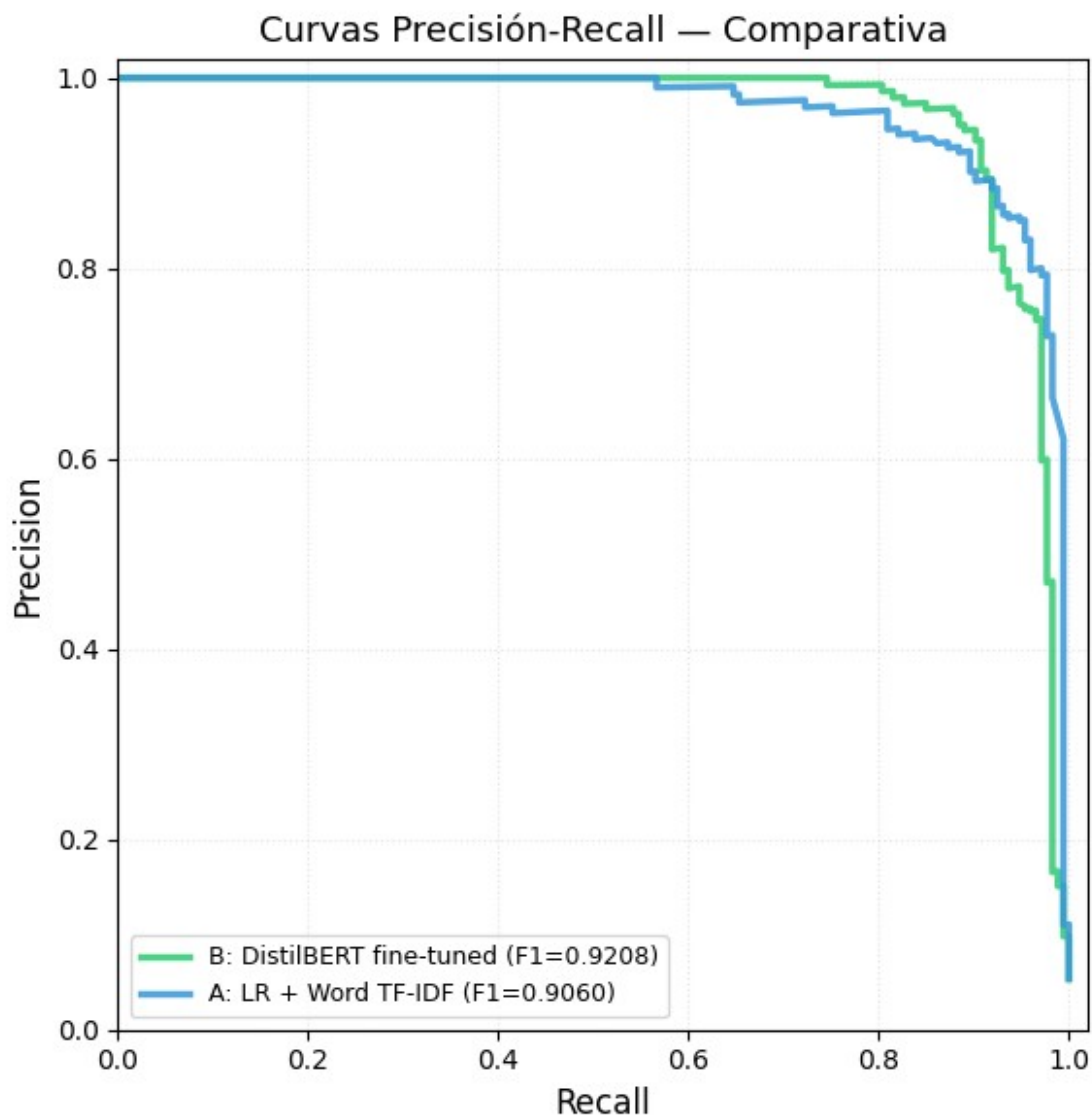


F1 score en el conjunto de TEST = 0.9208

```
# =====
# CURVAS PR COMPARATIVAS DE AMBOS MODELOS
# =====
fig, ax = plt.subplots(figsize=(7, 6))
ax.set_aspect('equal', adjustable='box')

colors = ['#2ecc71', '#3498db']
for idx, r in enumerate(sorted(ALL_RESULTS, key=lambda x: x['f1'],
reverse=True)):
    if r['probs'] is not None:
        p, rec, _ = precision_recall_curve(Y_test, r['probs'])
        ax.plot(rec, p, lw=2.5,
                label=f"{r['name']} (F1={r['f1']:.4f})",
                color=colors[idx % len(colors)], alpha=0.85)

ax.set_xlabel('Recall', fontsize=12)
ax.set_ylabel('Precision', fontsize=12)
ax.set_title('Curvas Precisión-Recall — Comparativa', fontsize=13)
ax.legend(loc='lower left', fontsize=9)
ax.grid(alpha=0.3, linestyle='dotted')
ax.set_xlim([0, 1.02])
ax.set_ylim([0, 1.02])
plt.tight_layout()
plt.show()
```



```
# =====
# MATRICES DE CONFUSIÓN + DISTRIBUCIÓN DE PROBABILIDADES
# =====
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Matriz de confusión del mejor modelo
cm = confusion_matrix(Y_test, preds_test)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Crude', 'Crude'], yticklabels=['No
Crude', 'Crude'],
            ax=axes[0])
axes[0].set_title(f'Confusión - {best_result["name"]}\nF1={F1_score_test:.4f}', fontweight='bold')
axes[0].set_ylabel('Real')
axes[0].set_xlabel('Predicho')
```

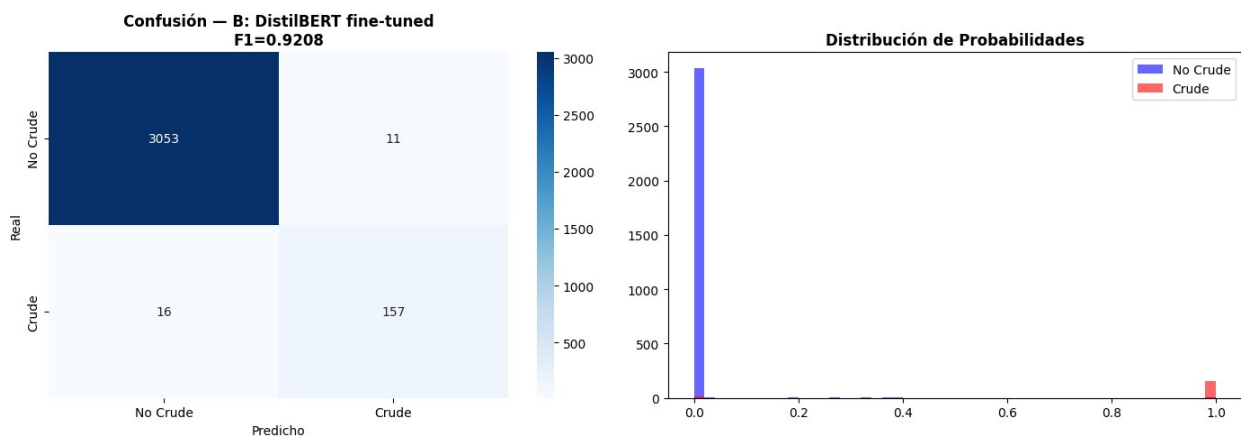
```

# Distribución de probabilidades
axes[1].hist(probs_best[Y_test == 0], bins=50, alpha=0.6, label='No
Crude', color='blue')
axes[1].hist(probs_best[Y_test == 1], bins=50, alpha=0.6,
label='Crude', color='red')
axes[1].set_title('Distribución de Probabilidades', fontweight='bold')
axes[1].legend()

plt.tight_layout()
plt.show()

# Análisis de errores
fn = (Y_test == 1) & (preds_test == 0)
fp = (Y_test == 0) & (preds_test == 1)
print(f"Falsos Negativos (crude no detectados): {fn.sum()}")
print(f"Falsos Positivos (no crude → crude): {fp.sum()}")
print(f"Total errores: {fn.sum() + fp.sum()} de {len(Y_test)}")

```



```

Falsos Negativos (crude no detectados): 16
Falsos Positivos (no crude → crude): 11
Total errores: 27 de 3237

```

```

# =====
# GRÁFICO BARRAS COMPARATIVO
# =====
fig, ax = plt.subplots(figsize=(10, 5))

sorted_r = sorted(ALL_RESULTS, key=lambda x: x['f1'])
names = [r['name'] for r in sorted_r]
f1s = [r['f1'] for r in sorted_r]
colors = ['#2ecc71' if f > 0.90 else '#3498db' for f in f1s]

bars = ax.barh(names, f1s, color=colors, edgecolor='black',
alpha=0.85, height=0.5)

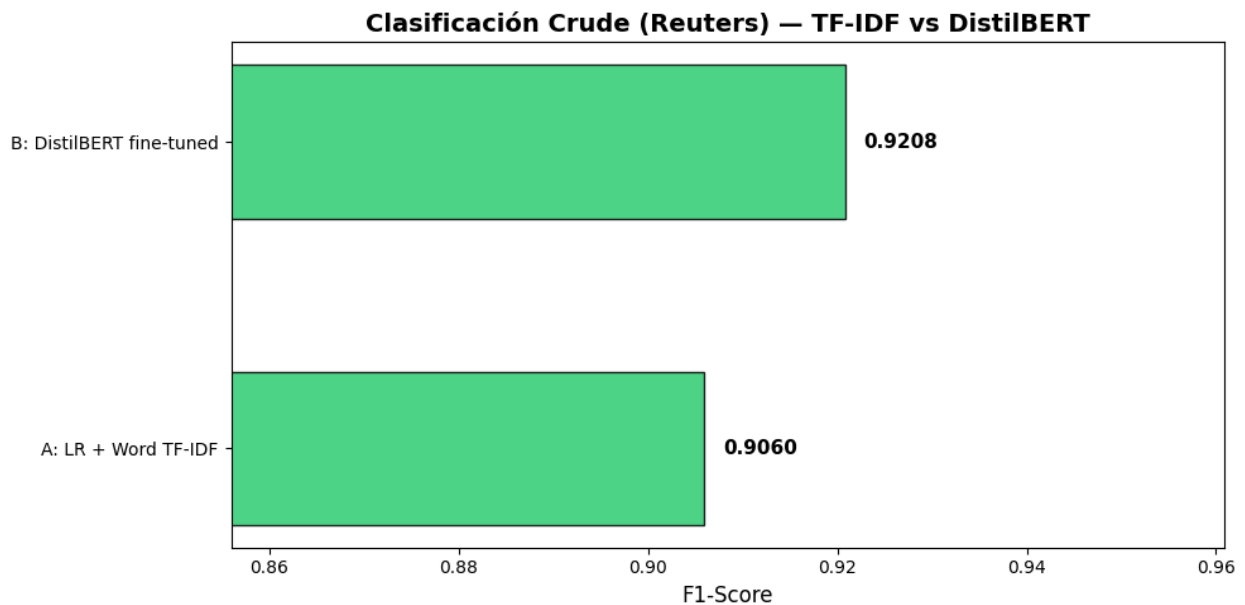
```

```

for bar, f1 in zip(bars, f1s):
    ax.text(bar.get_width() + 0.002, bar.get_y() + bar.get_height()/2,
            f'{f1:.4f}', va='center', fontweight='bold', fontsize=11)

ax.set_xlabel('F1-Score', fontsize=12)
ax.set_title('Clasificación Crude (Reuters) – TF-IDF vs DistilBERT',
            fontsize=14, fontweight='bold')
ax.set_xlim(min(f1s) - 0.05, max(f1s) + 0.04)
plt.tight_layout()
plt.show()

```



```

# =====
# CLASSIFICATION REPORT FINAL
# =====
print(f"\n{'='*60}")
print(f"RESULTADO FINAL: {best_result['name']}")
print(f"\n{'='*60}")
print(f"\nF1 score en el conjunto de TEST = {F1_score_test:.4f}")
print(f"\n{classification_report(Y_test, preds_test, target_names=['No Crude', 'Crude'])}")

```

```

=====
RESULTADO FINAL: B: DistilBERT fine-tuned
=====

```

F1 score en el conjunto de TEST = 0.9208

	precision	recall	f1-score	support
No Crude	0.99	1.00	1.00	3064

Crude	0.93	0.91	0.92	173
accuracy			0.99	3237
macro avg	0.96	0.95	0.96	3237
weighted avg	0.99	0.99	0.99	3237